helixqa_architecture

Below is the master analysis document for the **HelixQA** repository. It is organized exactly as requested in the prompt and is based on the actual source files (and repo metadata) pulled from `https://github.com/HelixDevelopment/HelixQA`. Each section contains concrete file paths, type/function signatures extracted from `raw.githubusercontent.com`, and architectural context.

HelixQA — Deep Architecture Analysis

> **Repository**: `https://github.com/HelixDevelopment/HelixQA`
> **License**: Apache-2.0
> **Primary language**: Go (97.6 %)
> **Commits examined**: ~570 (main branch)
> **Key external dependency**: `digital.vasic.challenges` (sibling repo under `../Challenges`)

---

1. Repository Layout (Top-Level)

Directory / File	Purpose
`cmd/helixqa/`	CLI entry point and subcommands
`pkg/`	40+ Go packages (see §2)
`banks/`	YAML/JSON test banks (see §6)
`challenges/`	Shell challenge scripts (see §7)
`tests/`	Unit, e2e, integration, stress, security, benchmark tests
`internal/visionserver/`	Internal vision-model HTTP server
`docker/`	Container definitions
`docs/`	Architecture & user-guide docs
`data/`	Vector-memory DB dumps
`monitoring/`	Metrics & dashboards
`tools/opensource/`	25+ Git submodules (see §12)
`web/src/capture/`	Web-based capture UI
`.env.example`	Master configuration template (see §9)
`go.mod`	Module: `digital.vasic.helixqa`

---

2. Complete Package Analysis (`pkg/`)

2.1 `pkg/orchestrator` — Main QA Brain

Responsibility: Central coordinator that loads test banks, iterates platforms, runs challenges via the *Challenges* runner, invokes step validation, and produces the combined report.

Key files: - `pkg/orchestrator/orchestrator.go` (196 lines, ~9.6 KB)

Key types / signatures:

```
type Result struct {
    Report     *reporter.QAReport `json:"report"`
    ReportPath string             `json:"report_path"`
    Success    bool               `json:"success"`
    StartTime  time.Time          `json:"start_time"`
    EndTime    time.Time          `json:"end_time"`
    Duration   time.Duration      `json:"duration"`
}

type Orchestrator struct {
    config   *config.Config
    detector *detector.Detector
    val      *validator.Validator
    reporter *reporter.Reporter
    logger   logging.Logger
    runner   runner.Runner
    bank     *bank.Bank
}

type Option func(*Orchestrator)

func New(cfg *config.Config, opts ...Option) *Orchestrator
func (o *Orchestrator) Run(ctx context.Context) (*Result, error)
func (o *Orchestrator) runPlatform(ctx context.Context, platform
        config.Platform, definitions []*challenge.Definition)
        (*reporter.PlatformResult, error)
```

Functional options (`WithLogger`, `WithRunner`, `WithDetector`, `WithValidator`, `WithReporter`, `WithBank`) allow dependency injection for testing.

Integration: - Imports `digital.vasic.challenges/pkg/{bank,challenge,runner,logging}` - Uses `pkg/detector` for crash checks, `pkg/validator` for step validation, `pkg/reporter` for output.

2.2 `pkg/testbank` — YAML Test Bank Management

Responsibility: Loads, validates, and filters QA-specific YAML/JSON test banks. Bridges to the *Challenges* `challenge.Definition` type for execution.

Key files: - `pkg/testbank/loader.go` - `pkg/testbank/manager.go` - `pkg/testbank/schema.go` - `pkg/testbank/generator.go`

Key types / signatures (`schema.go`):

```
type Priority string
const (
    PriorityCritical Priority = "critical"
    PriorityHigh     Priority = "high"
    PriorityMedium   Priority = "medium"
    PriorityLow      Priority = "low"
)

type TestCase struct {
    ID                 string           `yaml:"id" json:"id"`
    Name               string           `yaml:"name" json:"name"`
    Description        string           `yaml:"description"
        json:"description"`
    Category           string           `yaml:"category"
        json:"category"`
    Priority           Priority         `yaml:"priority"
        json:"priority"`
    Platforms          []config.Platform `yaml:"platforms"
        json:"platforms"`
    Steps              []TestStep       `yaml:"steps"
        json:"steps"`
    Dependencies       []string         `yaml:"dependencies"
        json:"dependencies"`
    DocumentationRefs []DocRef
        `yaml:"documentation_refs" json:"documentation_refs"`
    Tags               []string         `yaml:"tags" json:"tags"`
    EstimatedDuration string
        `yaml:"estimated_duration" json:"estimated_duration"`
    ExpectedResult     string           `yaml:"expected_result"
        json:"expected_result"`
    AllowForegroundLeave bool
        `yaml:"allow_foreground_leave,omitempty"`
    RequiresEnv        []string
        `yaml:"requires_env,omitempty"`
    _llm_driven        bool             // annotated on 41 banks
        per commit 57ccdc4
}

type TestStep struct {
    Name                string            `yaml:"name"
        json:"name"`
    Action              string            `yaml:"action"
        json:"action"`
```

```
    Expected           string             `yaml:"expected"
        json:"expected"`
    Platform           config.Platform
        `yaml:"platform,omitempty"`
    Timeout            int
        `yaml:"timeout,omitempty"`
    VisionVerify       bool
        `yaml:"vision_verify,omitempty"`
    Body               any                 `yaml:"body,omitempty"`
    Headers            map[string]string
        `yaml:"headers,omitempty"`
    AuthMode           string              `yaml:"auth,omitempty"`
    ExpectStatus       int
        `yaml:"expect_status,omitempty"`
    ExpectJSONPath     string
        `yaml:"expect_json_path,omitempty"`
    ExpectBodyContains string
        `yaml:"expect_body_contains,omitempty"`
    Skip               bool                `yaml:"_skip,omitempty"`
    SkipReason         string
        `yaml:"_skip_reason,omitempty"`
}

type BankFile struct {
    Version     string            `yaml:"version" json:"version"`
    Name        string            `yaml:"name" json:"name"`
    Description string            `yaml:"description"
        json:"description"`
    TestCases   []TestCase        `yaml:"test_cases"
        json:"test_cases"`
    Metadata    map[string]string `yaml:"metadata,omitempty"
        json:"metadata,omitempty"`
}

func LoadFile(path string) (*BankFile, error)
func (tc *TestCase) ToDefinition() *challenge.Definition
func (tc *TestCase) AppliesToPlatform(p config.Platform) bool
func (tc *TestCase) IsValid() string
func (ts *TestStep) ParseAction() (ActionType, string)
```

Action types (`ActionType` enum from `schema.go` lines 97-183): - `description`, `adb_shell`, `sleep`, `screenshot`, `keypress`, `tap`, `swipe`, `text`, `playback_check`, `frame_diff`, `http`, `assert`, `playwright`

Integration: - Converts `TestCase` → `challenge.Definition` via `ToDefinition()` - `pkg/testbank/loader.go` uses `gopkg.in/yaml.v3` for YAML and `encoding/json` for JSON; supports `"challenges"` as alternate key for `test_cases`.

2.3 `pkg/detector` — Real-Time Crash / ANR Detection

Responsibility: Per-platform crash and ANR detection.

Key file: `pkg/detector/detector.go` (~4.5 KB)

Key types:

```
type DetectionResult struct {
    Platform       config.Platform `json:"platform"`
    HasCrash       bool            `json:"has_crash"`
    HasANR         bool            `json:"has_anr"`
    ProcessAlive   bool            `json:"process_alive"`
    StackTrace     string          `json:"stack_trace,omitempty"`
    LogEntries     []string        `json:"log_entries,omitempty"`
    ScreenshotPath string
        `json:"screenshot_path,omitempty"`
    Timestamp      time.Time       `json:"timestamp"`
    Error          string          `json:"error,omitempty"`
}

type CommandRunner interface {
    Run(ctx context.Context, name string, args ...string)
        ([]byte, error)
}

type Detector struct { /* platform, device, packageName,
        browserURL, processName, processPID, evidenceDir,
        cmdRunner */ }

func New(platform config.Platform, opts ...Option) *Detector
func (d *Detector) Check(ctx context.Context) (*DetectionResult,
        error)
func (d *Detector) CheckApp(ctx context.Context, platform
        config.Platform) (*DetectionResult, error)
```

Platform-specific checks: - `checkAndroid` → ADB `logcat`, `pidof`, `screencap` - `checkWeb` → browser process monitoring, console error collection - `checkDesktop` → process alive checks, stderr monitoring

Integration: - Used by `pkg/validator` for pre/post-step validation. - `CommandRunner` interface enables mocking in tests.

2.4 `pkg/validator` — Step-by-Step Validation

Responsibility: Wraps the detector to perform pre/post-step validation with screenshot capture.

Key file: `pkg/validator/validator.go` (~6.2 KB)

Key types:

```
type StepStatus string
const ( StepPassed StepStatus = "passed"; StepFailed StepStatus
        = "failed"; StepSkipped StepStatus =
        "skipped"; StepError StepStatus = "error" )

type StepResult struct {
    StepName       string                  `json:"step_name"`
    Status         StepStatus              `json:"status"`
    Platform       config.Platform         `json:"platform"`
    Detection      *detector.DetectionResult
        `json:"detection,omitempty"`
    PreScreenshot  string
        `json:"pre_screenshot,omitempty"`
    PostScreenshot string
        `json:"post_screenshot,omitempty"`
    StartTime      time.Time               `json:"start_time"`
    EndTime        time.Time               `json:"end_time"`
    Duration       time.Duration           `json:"duration"`
    Error          string
        `json:"error,omitempty"`
}

type Validator struct { /* mu, det *detector.Detector,
        evidenceDir string */ }
```

Integration: - Called by `pkg/orchestrator` between each challenge step. - Produces `StepResult` with pre/post screenshots and detection correlation.

2.5 `pkg/evidence` — Evidence Collection

Responsibility: Centralized evidence gathering: screenshots, video, logcat, stack traces, console logs, audio.

Key file: `pkg/evidence/collector.go` (520 lines, 11.3 KB)

Key types:

```
type Type string
const (
    TypeScreenshot Type = "screenshot"
    TypeVideo      Type = "video"
    TypeLogcat     Type = "logcat"
    TypeStackTrace Type = "stacktrace"
    TypeConsoleLog Type = "console_log"
    TypeAudio      Type = "audio"
)

type Item struct {
    Type      Type           `json:"type"`
```

```
    Path      string          `json:"path"`
    Platform  config.Platform `json:"platform"`
    Step      string          `json:"step,omitempty"`
    Timestamp time.Time       `json:"timestamp"`
    Size      int64           `json:"size"`
}

type Collector struct {
    mu             sync.Mutex
    outputDir      string
    platform       config.Platform
    cmdRunner      detector.CommandRunner
    items          []Item
    recording      bool
    recordingID    string
    audioRecording bool
    audioRecordingID string
    audioCmd       *exec.Cmd
}

func New(opts ...Option) *Collector
func (c *Collector) CaptureScreenshot(ctx context.Context, name
        string) (*Item, error)
func (c *Collector) CaptureLogcat(ctx context.Context, name
        string, lines int) (*Item, error)
func (c *Collector) StartVideo(ctx context.Context) error
func (c *Collector) StopVideo(ctx context.Context) (*Item, error)
func (c *Collector) StartAudio(ctx context.Context) error
func (c *Collector) StopAudio(ctx context.Context) (*Item, error)
```

Platform-specific screenshot methods: - `captureAndroidScreenshot` → ADB `screencap -p` - `captureWebScreenshot` → Playwright / browser screenshot - `captureDesktopScreenshot` → X11 `import` or similar

Integration: - Used by `pkg/validator` for step screenshots. - Used by `pkg/session` for session-wide video/audio recording. - `pkg/evidence/annotator.go` adds visual annotations to screenshots.

2.6 `pkg/session` — Session Recording & Timeline

Responsibility: Coordinates video recording, screenshot capture, and timeline event tracking across platforms during an autonomous QA session.

Key files: - `pkg/session/recorder.go` (246 lines, 5.59 KB) - `pkg/session/timeline.go` - `pkg/session/video.go` - `pkg/session/cleanup.go`

Key types (`recorder.go`):

```
type Screenshot struct {
    Path        string        `json:"path"`
```

```
    Platform    string        `json:"platform"`
    Name        string        `json:"name"`
    Index       int           `json:"index"`
    Timestamp   time.Time     `json:"timestamp"`
    VideoOffset time.Duration `json:"video_offset"`
}

type SessionRecorder struct {
    sessionID     string
    outputDir     string
    videos        map[string]*VideoManager
    timeline      *Timeline
    screenshotIdx int
    mu            sync.Mutex
}

func NewSessionRecorder(sessionID, outputDir string)
        *SessionRecorder
func (sr *SessionRecorder) RecordScreenshot(platform string,
        data []byte) (*Screenshot, error)
func (sr *SessionRecorder) StartVideo(platform string) error
func (sr *SessionRecorder) StopVideo(platform string)
        (*VideoItem, error)
func (sr *SessionRecorder) RecordEvent(event TimelineEvent) error
func (sr *SessionRecorder) SessionID() string
```

Timeline event types (`timeline.go`):

```
type EventType string
const (
    EventAction    EventType = "action"
    EventDetection EventType = "detection"
    EventScreenshot EventType = "screenshot"
    EventVideo      EventType = "video"
    EventError      EventType = "error"
    EventPhase      EventType = "phase"
)

type TimelineEvent struct {
    Type        EventType     `json:"type"`
    Platform    string        `json:"platform,omitempty"`
    Description string        `json:"description"`
    Timestamp   time.Time     `json:"timestamp"`
    FeatureID   string        `json:"feature_id,omitempty"`
    Duration    time.Duration `json:"duration,omitempty"`
}
```

Integration: - Consumed by `pkg/evidence` for item storage. - Consumed by `pkg/autonomous` (PlatformWorker records events during doc-driven & curiosity phases). - `pkg/session/video.go` wraps `pkg/video` for per-platform video management.

2.7 `pkg/autonomous` — Autonomous QA Session

Responsibility: 4-phase autonomous QA session lifecycle orchestrated by `SessionCoordinator` and executed by parallel `PlatformWorkers`.

Key files (all in `pkg/autonomous/`):

File	Lines	Purpose
`coordinator.go`	467	`SessionCoordinator`, `SessionConfig`, phase orchestration
`worker.go`	~300	`PlatformWorker` — per-platform doc-driven + curiosity execution
`phase.go`	244	`PhaseManager`, `Phase`, `PhaseListener`, 4-phase definitions
`pipeline.go`	**3,958** (110 KB)	`SessionPipeline`, `PipelineConfig`, `PipelineResult` — master autonomous runner
`executor_factory.go`	120	`ExecutorFactory`, `DefaultExecutorFactory`, `NoopExecutorFactory`
`structured_executor.go`	~1,050	`StructuredTestExecutor` — runs bank test cases systematically
`http_executor.go`	~800	`HTTPExecutor` — executes `ActionTypeHTTP` steps with auth, CSRF, token cache
`playwright_executor.go`	~250	`PlaywrightExecutor` — browser automation for `ActionTypePlaywright`
`real_executor.go`	~200	Real-device executor wrappers
`screenshot.go`	153	`IsBlankScreenshot`, `resizeScreenshot` (480 px max for LLM)
`stagnation.go`	~200	`StagnationDetector` with BOCPD (Bayesian Online Change-Point Detection)
`bocpd.go`	~150	`BOCPD`, `BOCPDConfig` — per-frame change probability

File	Lines	Purpose
`geo_probe.go`	209	`ProbeGeoRestriction`, `GeoProbeResult` — geo-restriction probe via ADB+curl
`retry.go`	~100	Retry logic with exponential backoff
`sanitize.go`	~80	Input sanitization
`fallback.go`	~150	Fallback model selection
`adapters.go`	~200	External module adapters
`findings_bridge.go`	~150	Bridges findings to ticket generator
`device_preserve.go`	~100	Device state preservation
`result.go`	~80	`StepResult`, `SessionResult` types

4-Phase Lifecycle (`phase.go` lines 85-95):

```
func NewPhaseManager() *PhaseManager {
    return &PhaseManager{
        phases: []Phase{
            {Name: "setup",       Status: PhasePending},
            {Name: "doc-driven",  Status: PhasePending},
            {Name: "curiosity",   Status: PhasePending},
            {Name: "report",      Status: PhasePending},
        },
        current: -1,
    }
}
```

SessionCoordinator (`coordinator.go`):

```
type SessionCoordinator struct {
    config          *SessionConfig
    orchestrator    agent.AgentPool
    visionEngine    analyzer.Analyzer
    featureMap      *feature.FeatureMap
    executorFactory ExecutorFactory
    workers         map[string]*PlatformWorker
    phaseManager    *PhaseManager
    session         *session.SessionRecorder
    coverage        coverage.CoverageTracker
    status          SessionStatus
    mu              sync.Mutex
}

type SessionConfig struct {
    SessionID        string
```

```
    OutputDir         string
    Platforms         []string
    Timeout           time.Duration
    CoverageTarget    float64
    CuriosityEnabled  bool
    CuriosityTimeout  time.Duration
}
```

PlatformWorker (`worker.go`):

```
type PlatformWorker struct {
    platform      string
    agent         agent.Agent
    analyzer      analyzer.Analyzer
    navigator     *navigator.NavigationEngine
    issueDetector *issuedetector.IssueDetector
    coverage      coverage.CoverageTracker
    navGraph      graph.NavigationGraph
    session       *session.SessionRecorder
    executor      navigator.ActionExecutor
    mu            sync.Mutex
}

func (pw *PlatformWorker) RunDocDriven(ctx context.Context,
        features []feature.Feature) ([]StepResult, error)
func (pw *PlatformWorker) RunCuriosity(ctx context.Context,
        budget time.Duration) ([]StepResult, error)
```

PipelineConfig (`pipeline.go` lines 60-240) — key fields:

```
type PipelineConfig struct {
    ProjectRoot       string
    Platforms         []string
    OutputDir         string
    IssuesDir         string
    BanksDir          string
    HTTPBaseURL       string          // for ActionTypeHTTP
    PlaywrightCDPURL  string          // for ActionTypePlaywright
    Timeout           time.Duration
    PassNumber        int
    AndroidDevice     string
    AndroidPackage    string
    CompetingAppPackages []string     // Android TV foreign-app
        guard
    WebURL            string
    DesktopDisplay    string
    FFmpegPath        string
    CuriosityEnabled  bool
    CuriosityTimeout  time.Duration
```

```
    VisionHost       string
    VisionUser       string
    VisionModel      string
    UseLlamaCpp      bool
    LlamaCppModelPath string
    LlamaCppMMProjPath string
    QACredentials    map[string]string
    LlamaCppFreeGPU  bool
    VisionHosts      []string        // distributed vision
    VisionMultiUser  string
    LlamaCppRPCModelPath string
    ChatProviders    []llm.ProviderConfig
}
```

Integration: - Imports `digital.vasic.llmorchestrator/pkg/agent` (LLM agent pool) - Imports `digital.vasic.visionengine/pkg/{analyzer,graph,remote}` (vision engine) - Imports `digital.vasic.docprocessor/pkg/{coverage,feature}` (doc processor) - Uses `pkg/navigator`, `pkg/issuedetector`, `pkg/session`, `pkg/llm`, `pkg/planning`, `pkg/learning`, `pkg/memory`, `pkg/video`, `pkg/vision`, `pkg/performance`, `pkg/regression`, `pkg/reproduce`, `pkg/training`, `pkg/maestro`

2.8 `pkg/navigator` — Navigation Engine

Responsibility: Drives platform-specific UI interactions during autonomous QA. Bridges LLM agent decisions with physical UI actions.

Key files: - `pkg/navigator/engine.go` (~5.5 KB) - `pkg/navigator/executor.go` (~14 KB) - `pkg/navigator/api_executor.go` (~4.2 KB) - `pkg/navigator/cli_executor.go` (~3.3 KB) - `pkg/navigator/playwright_executor.go` (~8.2 KB) - `pkg/navigator/x11_executor.go` (~3.5 KB) - `pkg/navigator/llm_navigator.go` - `pkg/navigator/state.go` (~8.2 KB) - `pkg/navigator/dual_screen.go` - `pkg/navigator/tvkeyboard.go`

Key types (`engine.go`):

```
type NavigationEngine struct {
    agent    agent.Agent
    analyzer analyzer.Analyzer
    executor ActionExecutor
    graph    graph.NavigationGraph
    state    *StateTracker
}

func NewNavigationEngine(ag agent.Agent, az analyzer.Analyzer,
        exec ActionExecutor, navGraph graph.NavigationGraph)
        *NavigationEngine
func (ne *NavigationEngine) NavigateTo(ctx context.Context,
        target string) error
```

```
func (ne *NavigationEngine) PerformAction(ctx context.Context,
        action analyzer.Action) (*ActionResult, error)
func (ne *NavigationEngine) Explore(ctx context.Context, budget
        time.Duration) (*ExploreResult, error)
```

ActionExecutor interface (`executor.go`):

```
type ActionExecutor interface {
    Click(ctx context.Context, x, y int) error
    Type(ctx context.Context, text string) error
    Clear(ctx context.Context) error
    Scroll(ctx context.Context, direction string, amount int)
        error
    LongPress(ctx context.Context, x, y int) error
    Swipe(ctx context.Context, fromX, fromY, toX, toY int) error
    KeyPress(ctx context.Context, key string) error
    Back(ctx context.Context) error
    Home(ctx context.Context) error
    Screenshot(ctx context.Context) ([]byte, error)
}
```

Implementations: - `ADBExecutor` — Android via ADB (`adb shell input tap`, `input text`, `screencap`) - `PlaywrightExecutor` — Web via Playwright CDP / browser automation - `X11Executor` — Desktop Linux via X11 (`xte`, `import` for screenshots) - `APIExecutor` — REST API navigation - `CLIExecutor` — Command-line navigation

Integration: - Used by `pkg/autonomous/PlatformWorker` - `NavigationEngine` maintains a `graph.NavigationGraph` for path-finding and `StateTracker` for screen-state tracking.

2.9 `pkg/issuedetector` — LLM-Powered Issue Detection

Responsibility: Detects bugs via LLM vision analysis across visual, UX, accessibility, functional, performance, and crash categories.

Key files: - `pkg/issuedetector/detector.go` (~6.8 KB) - `pkg/issuedetector/categories.go` (~1.4 KB) - `pkg/issuedetector/llm_analyzer.go` (~7 KB) - `pkg/issuedetector/prompts.go` (~2 KB)

Key types (`detector.go`):

```
type IssueDetector struct {
    agent    agent.Agent
    analyzer analyzer.Analyzer
    session  *session.SessionRecorder
    mu       sync.Mutex
}
```

```
func NewIssueDetector(agent agent.Agent, analyzer
        analyzer.Analyzer, session *session.SessionRecorder)
        *IssueDetector
func (id *IssueDetector) AnalyzeScreen(ctx context.Context,
        screenshot []byte, platform string) ([]Issue, error)
func (id *IssueDetector) AnalyzeFlow(ctx context.Context,
        screenshots [][]byte, platform string) ([]Issue, error)
```

Issue categories (`categories.go`): - `CategoryVisual` — visual glitches, layout breaks - `CategoryUX` — usability problems - `CategoryAccessibility` — a11y violations - `CategoryFunctional` — broken functionality - `CategoryPerformance` — lag, jank, memory issues - `CategoryCrash` — crashes, ANRs

LLM Analyzer (`llm_analyzer.go`): - Sends screenshots to vision LLM with structured prompts (`prompts.go`) - Parses JSON responses into `Issue` structs - Supports severity scoring and confidence thresholds

Integration: - Called by `pkg/autonomous/PlatformWorker` after each action / exploration step. - Results feed into `pkg/ticket` for ticket generation.

2.10 `pkg/planning` — Test Planning

Responsibility: Generates and ranks test plans for autonomous sessions, with Android TV channel framework support.

Key files: - `pkg/planning/planner.go` - `pkg/planning/types.go` - `pkg/planning/ranker.go` - `pkg/planning/reconciler.go` - `pkg/planning/androidtv_channels_framework.go` - `pkg/planning/channels_support.go`

Integration: - Consumed by `pkg/autonomous/pipeline.go` for plan generation. - Uses `pkg/llm` for plan reasoning.

2.11 `pkg/llm` — LLM Provider Abstraction

Responsibility: Unified interface for multiple LLM providers with adaptive selection, cost tracking, rate limiting, consensus, and escalation.

Key files (30+ files): - `pkg/llm/provider.go` - `pkg/llm/adaptive.go` - `pkg/llm/bridge_provider.go` - `pkg/llm/escalation.go` - `pkg/llm/anthropic.go`, `openai.go`, `google.go`, `ollama.go`, `astica.go` - `pkg/llm/consensus.go`, `cost_tracker.go`, `ratelimiter.go`, `prompt_optimizer.go`, `vision_ranking.go`

Key types (`provider.go`):

```
type Provider interface {
    Name() string
    Chat(ctx context.Context, messages []Message) (Response,
        error)
    Vision(ctx context.Context, image []byte, prompt string)
        (Response, error)
```

```
    Health(ctx context.Context) error
}

type ProviderConfig struct {
    Name    string
    APIKey  string
    BaseURL string
    Model   string
}
```

Adaptive provider (`adaptive.go`): - Probes all configured providers and selects the best based on health, latency, and cost. - Supports dual-model architecture: vision model for screenshot analysis, chat model for reasoning/planning.

Integration: - Used throughout autonomous packages (`pkg/autonomous`, `pkg/navigator`, `pkg/issuedetector`, `pkg/planning`). - `pkg/llm/bridge_provider.go` bridges to external `digital.vasic.llmorchestrator` agents.

2.12 `pkg/vision` — Computer Vision & UI Detection

Responsibility: UI element detection, OCR, image diff, and LLM-powered vision analysis.

Key files: - `pkg/vision/detector.go` (682 lines, 16.7 KB) - `pkg/vision/diff.go` - `pkg/vision/llm_ollama.go` - `pkg/vision/ocr_paddle.go`, `ocr_tesseract.go`

Key types (`detector.go`):

```
type ElementType string
const (
    ElementButton   ElementType = "button"
    ElementInput    ElementType = "input"
    ElementText     ElementType = "text"
    ElementImage    ElementType = "image"
    ElementLink     ElementType = "link"
    ElementCheckbox ElementType = "checkbox"
    ElementRadio    ElementType = "radio"
    ElementDropdown ElementType = "dropdown"
    ElementSlider   ElementType = "slider"
    ElementToggle   ElementType = "toggle"
    ElementMenu     ElementType = "menu"
    ElementTab      ElementType = "tab"
    ElementScroll   ElementType = "scroll"
    ElementUnknown  ElementType = "unknown"
)

type Element struct {
    ID         string            `json:"id"`
    Type       ElementType       `json:"type"`
    Bounds     image.Rectangle   `json:"bounds"`
```

```
    Confidence float64           `json:"confidence"`
    Text       string            `json:"text,omitempty"`
    Label      string            `json:"label,omitempty"`
    Enabled    bool              `json:"enabled"`
    Visible    bool              `json:"visible"`
    Selected   bool              `json:"selected,omitempty"`
    Focused    bool              `json:"focused,omitempty"`
    Metadata   map[string]string `json:"metadata,omitempty"`
}

type FrameResult struct {
    FrameID    string      `json:"frame_id"`
    Timestamp  time.Time   `json:"timestamp"`
    Elements   []Element   `json:"elements"`
    TextBlocks []TextBlock `json:"text_blocks,omitempty"`
    LatencyMs  float64     `json:"latency_ms"`
}
```

Integration: - Used by `pkg/navigator` for UI element discovery. - Used by `pkg/issuedetector` for visual bug detection. - `pkg/vision/llm_ollama.go` interfaces with local Ollama vision models.

2.13 `pkg/capture` — Platform Capture Drivers

Responsibility: Low-level screenshot/video capture per platform.

Key files: - `pkg/capture/android_capture.go` - `pkg/capture/desktop_capture.go` - `pkg/capture/linux_capture.go` - `pkg/capture/macos_capture.go` - `pkg/capture/windows_capture.go`

Integration: - Called by `pkg/evidence/collector.go` for screenshots. - Called by `pkg/session/recorder.go` for session screenshots. - `pkg/capture/android_capture.go` uses scrcpy-direct path + v3 wire protocol.

2.14 `pkg/video` — Video Recording

Responsibility: Video recording via FFmpeg and scrcpy.

Key files: - `pkg/video/ffmpeg_recorder.go` - `pkg/video/scrcpy.go` - `pkg/video/frames.go`

Integration: - Used by `pkg/session/video.go` for per-platform video management. - Used by `pkg/evidence/collector.go` for video evidence.

2.15 `pkg/ticket` — Ticket Generation

Responsibility: Generates detailed Markdown issue tickets from failures.

Key file: `pkg/ticket/ticket.go` (~13.7 KB)

Key types:

```
type Severity string
const ( SeverityCritical Severity = "critical"; SeverityHigh
        Severity = "high"; SeverityMedium Severity = "medium";
        SeverityLow Severity = "low" )

type Ticket struct {
    ID               string            `json:"id"`
    Title            string            `json:"title"`
    Severity         Severity          `json:"severity"`
    Platform         config.Platform   `json:"platform"`
    Category         string            `json:"category"`
    Description      string            `json:"description"`
    ReproSteps       []string          `json:"repro_steps"`
    ExpectedBehavior string            `json:"expected_behavior"`
    ActualBehavior   string            `json:"actual_behavior"`
    StackTrace       string
        `json:"stack_trace,omitempty"`
    Logs             []string          `json:"logs,omitempty"`
    Screenshots      []string
        `json:"screenshots,omitempty"`
    VideoRef         *VideoReference
        `json:"video_ref,omitempty"`
    LLMSuggestedFix  *LLMSuggestedFix
        `json:"llm_suggested_fix,omitempty"`
    DocumentationRefs []validator.DocRef
        `json:"documentation_refs,omitempty"`
    Timestamp        time.Time         `json:"timestamp"`
}

func GenerateTicket(stepResult *validator.StepResult, detection
        *detector.DetectionResult) (*Ticket, error)
func WriteTicket(ticket *Ticket, path string) error
func ParseReplayScript(data []byte) ([]automation.Action,
        []string, error)
```

Integration: - Consumed by `pkg/reporter` for ticket output. - `cmd/helixqa/replay.go` uses `ticket.ParseReplayScript()` to replay `.ocu-replay` DSL blocks.

2.16 `pkg/reporter` — Report Generation

Responsibility: Produces QA reports in Markdown, HTML, and JSON.

Key file: `pkg/reporter/reporter.go` (~11 KB)

Key types:

```go
type QAReport struct {
    Title            string            `json:"title"`
    GeneratedAt      time.Time         `json:"generated_at"`
    PlatformResults  []*PlatformResult `json:"platform_results"`
    TotalChallenges  int               `json:"total_challenges"`
    PassedChallenges int               `json:"passed_challenges"`
    FailedChallenges int               `json:"failed_challenges"`
    TotalCrashes     int               `json:"total_crashes"`
    TotalANRs        int               `json:"total_anrs"`
    TotalDuration    time.Duration     `json:"total_duration"`
    OutputDir        string            `json:"output_dir"`
}

type Reporter struct {
    challengeReporter report.Reporter
    outputDir         string
    reportFormat      config.ReportFormat
}

func (r *Reporter) GenerateQAReport(results []*PlatformResult)
        (*QAReport, error)
func (r *Reporter) WriteMarkdown(qa *QAReport, path string) error
func (r *Reporter) WriteJSON(qa *QAReport, path string) error
func (r *Reporter) WriteHTML(qa *QAReport, path string) error
```

Integration: - Reuses `digital.vasic.challenges/pkg/report` for challenge-level formatting. - Called by `pkg/orchestrator` at the end of a run.

2.17 `pkg/config` — Configuration

Responsibility: Configuration types and parsing.

Key file: `pkg/config/config.go` (~14 KB)

Key types:

```go
type Platform string
const (
    PlatformAndroid    Platform = "android"
    PlatformAndroidTV  Platform = "androidtv"
    PlatformWeb        Platform = "web"
    PlatformDesktop    Platform = "desktop"
    PlatformCLI        Platform = "cli"
    PlatformAPI        Platform = "api"
    PlatformAll        Platform = "all"
)

type SpeedMode string
```

```
const ( SpeedSlow SpeedMode = "slow"; SpeedNormal SpeedMode =
        "normal"; SpeedFast SpeedMode = "fast" )

type ReportFormat string
const ( ReportMarkdown ReportFormat = "markdown"; ReportHTML
        ReportFormat = "html"; ReportJSON ReportFormat = "json" )

type Config struct {
    Banks          []string        `yaml:"banks" json:"banks"`
    Platforms      []Platform      `yaml:"platforms"
        json:"platforms"`
    Device         string          `yaml:"device" json:"device"`
    PackageName    string          `yaml:"package_name"
        json:"package_name"`
    OutputDir      string          `yaml:"output_dir"
        json:"output_dir"`
    Speed          SpeedMode       `yaml:"speed" json:"speed"`
    ReportFormat   ReportFormat    `yaml:"report_format"
        json:"report_format"`
    ValidateSteps  bool            `yaml:"validate"
        json:"validate"`
    Record         bool            `yaml:"record" json:"record"`
    Verbose        bool            `yaml:"verbose"
        json:"verbose"`
    Timeout        time.Duration   `yaml:"timeout"
        json:"timeout"`
    StepTimeout    time.Duration   `yaml:"step_timeout"
        json:"step_timeout"`
    BrowserURL     string          `yaml:"browser_url"
        json:"browser_url"`
    DesktopProcess string          `yaml:"desktop_process"
        json:"desktop_process"`
    DesktopPID     int             `yaml:"desktop_pid"
        json:"desktop_pid"`
    Autonomous     AutonomousConfig `yaml:"autonomous"
        json:"autonomous"`
}

type AutonomousConfig struct {
    Enabled          bool            `yaml:"enabled"
        json:"enabled"`
    CoverageTarget   float64         `yaml:"coverage_target"
        json:"coverage_target"`
    CuriosityEnabled bool            `yaml:"curiosity_enabled"
        json:"curiosity_enabled"`
    CuriosityTimeout time.Duration   `yaml:"curiosity_timeout"
        json:"curiosity_timeout"`
```

```
	AgentsEnabled    []string        `yaml:"agents_enabled"
	    json:"agents_enabled"`
	AgentPoolSize    int             `yaml:"agent_pool_size"
	    json:"agent_pool_size"`
	AgentTimeout     time.Duration   `yaml:"agent_timeout"
	    json:"agent_timeout"`
	AgentMaxRetries  int             `yaml:"agent_max_retries"
	    json:"agent_max_retries"`
	VisionProvider   string          `yaml:"vision_provider"
	    json:"vision_provider"`
	VisionOpenCVEnabled bool
	    `yaml:"vision_opencv_enabled"
	    json:"vision_opencv_enabled"`
	VisionSSIMThreshold float64
	    `yaml:"vision_ssim_threshold"
	    json:"vision_ssim_threshold"`
	DocsRoot         string          `yaml:"docs_root"
	    json:"docs_root"`
	DocsAutoDiscover bool            `yaml:"docs_auto_discover"
	    json:"docs_auto_discover"`
	DocsFormats      []string        `yaml:"docs_formats"
	    json:"docs_formats"`
	RecordingVideo           bool    `yaml:"recording_video"
	    json:"recording_video"`
	RecordingScreenshots     bool
	    `yaml:"recording_screenshots"
	    json:"recording_screenshots"`
	RecordingVideoQuality    string
	    `yaml:"recording_video_quality"
	    json:"recording_video_quality"`
	RecordingScreenshotFormat string
	    `yaml:"recording_screenshot_format"
	    json:"recording_screenshot_format"`
	RecordingAudio           bool    `yaml:"recording_audio"
	    json:"recording_audio"`
	RecordingAudioQuality    string
	    `yaml:"recording_audio_quality"
	    json:"recording_audio_quality"`
	RecordingAudioFormat     string
	    `yaml:"recording_audio_format"
	    json:"recording_audio_format"`
	RecordingAudioDevice     string
	    `yaml:"recording_audio_device"
	    json:"recording_audio_device"`
	RecordingFFmpegPath      string
	    `yaml:"recording_ffmpeg_path"
	    json:"recording_ffmpeg_path"`
```

```
    AndroidDevice    string          `yaml:"android_device"
        json:"android_device"`
    AndroidPackage   string          `yaml:"android_package"
        json:"android_package"`
    WebURL           string          `yaml:"web_url"
        json:"web_url"`
    WebBrowser       string          `yaml:"web_browser"
        json:"web_browser"`
    DesktopProcess   string          `yaml:"desktop_process"
        json:"desktop_process"`
    DesktopDisplay   string          `yaml:"desktop_display"
        json:"desktop_display"`
    ReportFormats    []string        `yaml:"report_formats"
        json:"report_formats"`
    TicketsEnabled   bool            `yaml:"tickets_enabled"
        json:"tickets_enabled"`
    TicketsMinSeverity string
        `yaml:"tickets_min_severity" json:"tickets_min_severity"`
    LLMProvider      string          `yaml:"llm_provider"
        json:"llm_provider"`
    LLMAPIKey        string          `yaml:"llm_api_key"
        json:"llm_api_key"`
    LLMBaseURL       string          `yaml:"llm_base_url"
        json:"llm_base_url"`
    LLMModel         string          `yaml:"llm_model"
        json:"llm_model"`
}

func ParsePlatforms(s string) ([]Platform, error)
func ParseBanks(s string) []string
func (c *Config) Validate() error
```

2.18 Other Packages (Summary)

Package	Key Files	Responsibility
`pkg/controller`	`controller.go`	Process lifecycle management, fallback model control
`pkg/bridge`	`dbusportal/`, `scrcpy/`, `sidecarutil/`	D-Bus portal, scrcpy server lifecycle, sidecar utilities
`pkg/bridges`	`registry.go`	HelixQA-native sidecar probe registry
`pkg/discovery`	`host_discovery.go`	Host discovery for distributed vision
`pkg/distributed`	`state.go`	Distributed state management

Package	Key Files	Responsibility
pkg/learning	knowledge.go, codebase.go, manifest.go, platform_features.go	Learn phase: discovers project structure, credentials, features
pkg/memory	store.go, findings.go, cognitive.go, coverage.go, sessions.go	Vector-memory DB, cognitive memory, session persistence
pkg/maestro	maestro.go	Maestro FlowRunner for YAML mobile flow execution
pkg/performance	collector.go, exec.go, types.go	Performance metrics collection
pkg/regression	visual.go, pixelmatch.go, deltae.go, report.go	Visual regression testing (pixel match, Delta E)
pkg/replay	buffer.go	Replay buffer management
pkg/reproduce	reproducer.go	Issue reproduction orchestration
pkg/streaming	scrcpy_rtsp_bridge.go, webrtc_handler.go, webrtc_server.go	Scrcpy RTSP bridge, WebRTC streaming
pkg/types	issue.go	Common issue types
pkg/validators	image.go, text.go, video.go, manager.go, types.go	Validators for image, text, video assets
pkg/visionnav	provider.go, explorer.go, evidence.go	Vision-nav provider interface + NopProvider
pkg/visual	comparator.go	Visual comparator
pkg/audio	stream.go, tesseract_client.go, whisper_client.go	Audio stream processing, speech-to-text
pkg/agent	action/, explore/, graph/, ground/	Agent action primitives, exploration, graph, grounding
pkg/analysis	pelt.go	PELT change-point segmentation
pkg/nexus	adapter.go, doc.go	Nexus adapter for cross-project integration
pkg/observe/frida	frida bridge	Frida dynamic instrumentation HTTP bridge
pkg/opensource	Various	Open-source tool wrappers
pkg/infra	Various	Infrastructure helpers
pkg/training	Various	Training data management

3. CLI Commands (`cmd/helixqa/`)

Entry point: `cmd/helixqa/main.go` (version `0.2.0`)

Subcommands:

Subcommand	Flags	Implementation File	Purpose
`run`	`--banks`, `--platform`, `--device`, `--output`, `--speed`, `--report`, `--validate`, `--record`, `--verbose`, `--package`, `--timeout`, `--browser-url`, `--desktop-process`, `--tickets`	`main.go` lines 94-212	Execute QA pipeline
`list`	`--banks`, `--platform`, `--category`, `--priority`, `--tag`, `--json`	`main.go` lines 214-304	List/filter test cases
`report`	`--input`, `--format`, `--output`	`main.go` lines 306+	Generate report from existing results
`autonomous`	`--project`, `--platforms`, `--env`, `--timeout`	`main.go` (calls `autonomous` package)	Run autonomous LLM-driven QA session
`replay`	`--ticket` (required), `--execute`	`cmd/helixqa/replay.go`	Replay a ticket's OCU action chain (dry-run by default)
`version`	—	`main.go` line 65	Print version

Subcommand	Flags	Implementation File	Purpose
`help` / `-h` / `--help`	—	`main.go` line 76	Show usage

Additional CLIs in `cmd/` directory: - `helixqa-axtree-darwin/` — macOS accessibility tree extractor - `helixqa-axtree-windows/` — Windows accessibility tree extractor - `helixqa-capture-demo/` — Capture demo tool - `helixqa-capture-linux/` — Linux capture helper - `helixqa-dreamsim/` — DreamSim perceptual similarity - `helixqa-frida-bridge/` — Frida bridge CLI - `helixqa-input/` — Input injection tool - `helixqa-kmsgrab/` — KMS grab tool - `helixqa-lpips/` — LPIPS perceptual similarity - `helixqa-omniparser/` — OmniParser CLI - `helixqa-text/` — Text extraction CLI - `helixqa-uitars/` — UI-TARS CLI - `helixqa-x11grab/` — X11 grab tool - `ocu-dispatch-test/` — OCU dispatch test - `ocu-probe/` — OCU probe - `qa-audio-probe/` — Audio probe

---

4. Configuration System (`pkg/config/` + `.env.example`)

4.1 `.env.example` Configuration Options

Section	Key	Default
Master Switch	`HELIX_AUTONOMOUS_ENABLED`	`true`
	`HELIX_AUTONOMOUS_PLATFORMS`	`android,desktop,web`
	`HELIX_AUTONOMOUS_TIMEOUT`	`2h`
	`HELIX_AUTONOMOUS_COVERAGE_TARGET`	`0.90`
	`HELIX_AUTONOMOUS_CURIOSITY_ENABLED`	`true`
	`HELIX_AUTONOMOUS_CURIOSITY_TIMEOUT`	`30m`
LLMsVerifier	`LLMSVERIFIER_CONFIG`	`./llmsverifier.yaml`
	`LLMSVERIFIER_STRATEGY`	`helix-qa`
	`LLMSVERIFIER_MIN_SCORE`	`0.6`
	`LLMSVERIFIER_MAX_MODELS`	`5`
	`LLMSVERIFIER_CACHE_RESULTS`	`true`
	`LLMSVERIFIER_CACHE_TTL`	`24h`

Section	Key	Default
Distributed Vision	HELIX_VISION_HOSTS	thinker.local,amber.local
	HELIX_VISION_MULTI_USER	milosvasic
	HELIX_LLAMACPP_RPC_MODEL	~/models/vision-model.gguf
Vision Providers	ASTICA_API_KEY	—
	OPENAI_API_KEY	—
	ANTHROPIC_API_KEY	—
	GEMINI_API_KEY	—
	KIMI_API_KEY	—
	STEPFUN_API_KEY	—
	NVIDIA_API_KEY	—
	GROQ_API_KEY	—
	MISTRAL_API_KEY	—
	DEEPSEEK_API_KEY	—
	XAI_API_KEY	—
	TOGETHER_API_KEY	—
	QWEN_API_KEY	—
	JUNIE_API_KEY	—
Local Vision	HELIX_OLLAMA_URL	http://thinker.local:11434
	HELIX_OLLAMA_MODEL	minicpm-v:8b
CLI Agents	HELIX_AGENTS_ENABLED	opencode,claude-code,gemini
	HELIX_AGENT_*_PATH	various
	HELIX_AGENT_TIMEOUT	60s
	HELIX_AGENT_MAX_RETRIES	3
	HELIX_AGENT_POOL_SIZE	3
Vision Engine	HELIX_VISION_PROVIDER	auto
	HELIX_VISION_OPENCV_ENABLED	true

Section	Key	Default
	HELIX_VISION_SSIM_THRESHOLD	0.95
	HELIX_VISION_MAX_IMAGE_SIZE	4096
Doc Processor	HELIX_DOCS_ROOT	./docs
	HELIX_DOCS_AUTO_DISCOVER	true
	HELIX_DOCS_FORMATS	md,yaml,html,adoc,rst
Recording	HELIX_RECORDING_VIDEO	true
	HELIX_RECORDING_SCREENSHOTS	true
	HELIX_RECORDING_VIDEO_QUALITY	medium
	HELIX_RECORDING_SCREENSHOT_FORMAT	png
	HELIX_RECORDING_FFMPEG_PATH	/usr/bin/ffmpeg
Audio Recording	HELIXQA_RECORDING_AUDIO	false
	HELIXQA_RECORDING_AUDIO_QUALITY	high
	HELIXQA_RECORDING_AUDIO_FORMAT	wav
	HELIXQA_RECORDING_AUDIO_DEVICE	default
Platform	HELIX_ANDROID_DEVICE	emulator-5554
	HELIX_ANDROID_PACKAGE	com.example.app
	HELIX_WEB_URL	http://localhost:8080
	HELIX_WEB_BROWSER	chromium
	HELIX_DESKTOP_PROCESS	yole-desktop
	HELIX_DESKTOP_DISPLAY	:0
Output	HELIX_OUTPUT_DIR	./qa-results
	HELIX_REPORT_FORMATS	markdown,html,json
	HELIX_TICKETS_ENABLED	true
	HELIX_TICKETS_MIN_SEVERITY	low

Section	Key	Default
Remote Vision	`HELIX_VISION_HOST`	—
	`HELIX_VISION_USER`	—
	`HELIX_VISION_MODEL`	`llava:7b`
Distributed RPC	`HELIX_LLAMACPP_RPC_ENABLED`	`false`
	`HELIX_LLAMACPP_RPC_WORKERS`	—
Device Exclusion	`HELIX_ADB_EXCLUDE`	`ATMOSphere`

---

5. Test Bank System (`banks/`)

5.1 Bank File Count & Types

The `banks/` directory contains **~120 bank files** in both YAML and JSON formats (many pairs: `.yaml` + `.json`).

Complete file listing (from `api.github.com` listing, 2026-04-30):

Android / Mobile: - `full-qa-android.yaml` / `.json` — Comprehensive Android phone QA (106 KB YAML) - `full-qa-androidtv.yaml` / `.json` — Android TV comprehensive QA (166 KB YAML) - `nexus-mobile-android.yaml` / `.json` — Nexus mobile Android - `nexus-mobile-ios.yaml` / `.json` — Nexus mobile iOS - `capture-android.yaml` — Android capture integration tests - `validation-androidtv-focus.json` — Android TV focus validation

Web / API: - `full-qa-web.yaml` / `.json` — Comprehensive web QA (172 KB YAML) - `full-qa-api.yaml` / `.json` — API endpoint QA - `nexus-browser.yaml` / `.json` — Browser-specific Nexus tests - `nexus-a11y.yaml` / `.json` — Accessibility tests

Desktop: - `nexus-desktop-linux.yaml` / `.json` — Linux desktop - `nexus-desktop-macos.yaml` / `.json` — macOS desktop - `nexus-desktop-windows.yaml` / `.json` — Windows desktop - `capture-linux.yaml` — Linux capture tests

Cross-Platform / Comprehensive: - `full-qa-cross-platform.yaml` / `.json` - `all-formats.yaml` / `.json` - `edge-cases-stress.yaml` / `.json`

Feature-Specific: - `atmosphere.yaml` / `.json` - `app-navigation.yaml` / `.json` - `file-browser.yaml` / `.json` - `editor-operations.yaml` / `.json` - `entity-management.yaml` / `.json` - `cloud-storage-operations.yaml` / `.json` - `storage-configuration.yaml` / `.json` - `admin-operations.yaml` / `.json` - `image-quality-gate.yaml` / `.json` - `performance-validation.yaml` / `.json` - `security-validation.yaml` / `.json` - `security-comprehensive.yaml` - `ddos-ratelimit-comprehensive.yaml` - `benchmarking-baselines.yaml`

CLI / Agent: - `cli-agents-comprehensive.yaml` / `.json` - `cli-agents-test-helixagent.yaml` / `.json` - `cli-agent-e2e-flow.yaml` - `helixagent-cli-agent-tests.yaml` - `aichat-bash-tools-comprehensive.yaml` / `.json`

Fixes Validation (13 banks): - `fixes-validation.yaml` / `.json` - `fixes-validation-a11y.yaml` / `.json` - `fixes-validation-ai.yaml` / `.json` - `fixes-validation-browser.yaml` / `.json` - `fixes-validation-cover.yaml` / `.json` - `fixes-validation-decoupling.yaml` / `.json` - `fixes-validation-desktop.yaml` / `.json` - `fixes-validation-mobile.yaml` / `.json` - `fixes-validation-obs.yaml` / `.json` - `fixes-validation-perf.yaml` / `.json` - `fixes-validation-xflow.yaml` / `.json`

Nexus (10 banks): - `nexus-ai.yaml` / `.json` - `nexus-browser.yaml` / `.json` - `nexus-desktop-linux.yaml` / `.json` - `nexus-desktop-macos.yaml` / `.json` - `nexus-desktop-windows.yaml` / `.json` - `nexus-mobile-android.yaml` / `.json` - `nexus-mobile-ios.yaml` / `.json` - `nexus-observability.yaml` / `.json` - `nexus-perf.yaml` / `.json` - `nexus-xflow.yaml` / `.json`

OCU (OpenClawing): - `ocu-adversarial.json` - `ocu-automation.json` - `ocu-capture.json` - `ocu-cross-platform.json` - `ocu-fixes-validation.json` - `ocu-foundation.json` - `ocu-interact.json` - `ocu-observe.json` - `ocu-record.json` - `ocu-tickets.json` - `ocu-vision.json`

OpenClawing2 (Phase references): - `openclawing2-phase1-references.yaml` / `.json` - `openclawing2-phase2-agent-step.yaml` / `.json` - `openclawing2-phase3-message-manager.yaml` / `.json` - `openclawing2-phase4-retry-healer-loop.yaml` / `.json` - `openclawing2-phase5-primitives.yaml` / `.json` - `openclawing2-phase6-coord-actions.yaml` / `.json` - `openclawing2-phase7-rich-ticketing.yaml` / `.json`

Phase / GoCore: - `phase1-gocore.yaml` (45 KB) - `phase2-gocore.yaml` - `phase3-gocore.yaml` - `phase6-gocore.yaml`

Other: - `docs-audit.yaml` - `input-linux.yaml` - `openclawing2-phase1-references.json`

5.2 YAML Structure

```yaml
version: "1.0"
name: "Catalogizer Full QA - Android Phone"
description: "Comprehensive test bank..."
metadata:
  author: "vasic-digital"
  app: "Catalogizer"
  version: "2.3.0"

test_cases:
  - id: FQA-AND-001
    name: "Cold launch displays login screen"
    category: functional
    priority: critical
    platforms: [android]
```

```yaml
    steps:
      - name: "Launch app from launcher"
        action: "Tap Catalogizer icon on home screen or app
         drawer"
        expected: "Splash screen appears briefly, then login
         screen is displayed"
    tags: [launch, login, cold-start, ui]
    estimated_duration: "10s"
    expected_result: "App cold-launches to login screen with all
         UI elements visible"
    _llm_driven: true   # on 41 banks per commit 57ccdc4
```

5.3 Bank Loading Machinery

- `pkg/testbank/loader.go`: `LoadFile(path)` supports `.yaml` and `.json`. JSON files accept `"challenges"` as alternate key for `"test_cases"`.
- `pkg/testbank/manager.go`: `Manager` holds loaded banks; supports filtering by platform, category, priority, tag.
- Validation: duplicate ID detection, `IsValid()` checks, intra-bank uniqueness enforcement.

---

6. Challenge System (`challenges/`)

6.1 Directory Structure

```
challenges/
├── config/
└── scripts/
    ├── host_no_auto_suspend_challenge.sh
    └── no_suspend_calls_challenge.sh
```

6.2 Challenge Scripts

`challenges/scripts/host_no_auto_suspend_challenge.sh` (3.7 KB): - Validates that the host system does not auto-suspend during QA sessions. - Checks systemd sleep settings, AC power policy, and screensaver inhibition. - Returns `PASS`/`FAIL` with descriptive messages.

`challenges/scripts/no_suspend_calls_challenge.sh`: - Validates that no suspend/hibernate calls are made by the test harness. - Uses `strace`-style monitoring or log analysis to detect `suspend`, `hibernate`, `pm-suspend` invocations.

Integration: - Challenge scripts are executed by the *Challenges* framework (`digital.vasic.challenges/pkg/runner`). - HelixQA loads challenge definitions from `banks/` and delegates execution to the Challenges runner. - Results flow back through `pkg/orchestrator` → `pkg/reporter`.

---

7. Autonomous Session Deep Dive

7.1 Architecture Overview

The autonomous QA session extends HelixQA with LLM-powered autonomous testing. A `SessionCoordinator` manages 4 sequential phases, delegating platform testing to parallel `PlatformWorker` instances.

External module dependencies (Git submodules): - `LLMsVerifier` — strategy pattern, model scoring - `LLMOrchestrator` — agent pool, CLI adapters - `VisionEngine` — GoCV + LLM Vision, NavigationGraph - `DocProcessor` — feature maps, coverage tracking

7.2 4-Phase Lifecycle

Phase	File	Description
1. Setup	`pkg/autonomous/coordinator.go`	Initialize agents, vision engine, executor factory, session recorder
2. Doc-Driven	`pkg/autonomous/worker.go RunDocDriven()`	Execute documented features from `DocProcessor.FeatureMap`; verify each feature's test steps
3. Curiosity	`pkg/autonomous/worker.go RunCuriosity()`	LLM-driven exploration beyond documented features; budget-limited (`CuriosityTimeout`)
4. Report	`pkg/autonomous/pipeline.go`	Generate findings report, tickets, coverage analysis

Phase Manager (`pkg/autonomous/phase.go`):

```
type PhaseStatus string
const ( PhasePending PhaseStatus = "pending"; PhaseRunning
        PhaseStatus = "running"; PhaseCompleted PhaseStatus =
        "completed"; PhaseFailed PhaseStatus = "failed";
        PhaseSkipped PhaseStatus = "skipped" )

type Phase struct {
    Name      string      `json:"name"`
    Status    PhaseStatus `json:"status"`
    StartAt   time.Time   `json:"start_at,omitempty"`
    EndAt     time.Time   `json:"end_at,omitempty"`
    Progress  float64     `json:"progress"`
    Error     error       `json:"-"`
}

type PhaseManager struct {
    phases    []Phase
```

```
    current   int
    listeners []PhaseListener
    mu        sync.Mutex
}
```

7.3 PlatformWorker Execution

```
// Per-platform doc-driven testing
func (pw *PlatformWorker) RunDocDriven(ctx context.Context,
        features []feature.Feature) ([]StepResult, error)

// Per-platform curiosity-driven exploration
func (pw *PlatformWorker) RunCuriosity(ctx context.Context,
        budget time.Duration) ([]StepResult, error)
```

Worker internals: - Gets its own `agent.Agent` (LLM), `analyzer.Analyzer` (vision), `navigator.ActionExecutor` (platform-specific), `issuedetector.IssueDetector`. - Records all events via `session.SessionRecorder`. - `NavigationEngine` (`pkg/navigator/engine.go`) performs path-finding and action execution. - `StagnationDetector` (`pkg/autonomous/stagnation.go`) detects UI freeze using BOCPD (Bayesian Online Change-Point Detection) over dHash Hamming-distance stream.

7.4 NavigationEngine

```
type NavigationEngine struct {
    agent    agent.Agent
    analyzer analyzer.Analyzer
    executor ActionExecutor
    graph    graph.NavigationGraph
    state    *StateTracker
}

func (ne *NavigationEngine) NavigateTo(ctx context.Context,
        target string) error
func (ne *NavigationEngine) PerformAction(ctx context.Context,
        action analyzer.Action) (*ActionResult, error)
func (ne *NavigationEngine) Explore(ctx context.Context, budget
        time.Duration) (*ExploreResult, error)
```

Actions supported: `click`, `type`, `scroll`, `long_press`, `swipe`, `key_press`, `back`, `home`

Screen-change detection: `StateTracker` tracks current screen; compares pre/post screenshots to determine `ScreenChanged`.

7.5 IssueDetector (LLM Bug Detection)

```
type IssueDetector struct {
    agent    agent.Agent
    analyzer analyzer.Analyzer
```

```
    session  *session.SessionRecorder
}

func (id *IssueDetector) AnalyzeScreen(ctx context.Context,
        screenshot []byte, platform string) ([]Issue, error)
func (id *IssueDetector) AnalyzeFlow(ctx context.Context,
        screenshots [][]byte, platform string) ([]Issue, error)
```

Detection categories (`pkg/issuedetector/categories.go`): - `CategoryVisual` — layout breaks, rendering artifacts - `CategoryUX` — confusing flows, missing feedback - `CategoryAccessibility` — missing labels, contrast issues, screen-reader problems - `CategoryFunctional` — buttons not working, state not persisting - `CategoryPerformance` — jank, memory growth, slow transitions - `CategoryCrash` — crashes, ANRs, exceptions

LLM prompt flow (`pkg/issuedetector/prompts.go`): 1. Encode screenshot as base64 or downscaled PNG (max 480 px wide for speed). 2. Send to vision LLM with structured prompt requesting JSON output. 3. Parse JSON into `Issue` structs with severity, confidence, category, description. 4. Filter by confidence threshold; deduplicate against existing findings.

7.6 SessionRecorder & Timeline

```
type SessionRecorder struct {
    sessionID     string
    outputDir     string
    videos        map[string]*VideoManager
    timeline      *Timeline
    screenshotIdx int
    mu            sync.Mutex
}
```

Video recording: - Per-platform `VideoManager` starts/stops FFmpeg or scrcpy recording. - `StartVideo(platform)` → `StopVideo(platform)` lifecycle. - Screenshots are indexed and correlated with video offset (`VideoOffset`).

Timeline: - `TimelineEvent` records every action, detection, screenshot, video segment, error, and phase transition. - Events are JSON-serializable and feed into the final report.

---

8. Screenshot & Evidence Pipeline

8.1 Screenshot Capture Per Platform

Platform	Method	File
Android	ADB `screencap -p`	`pkg/evidence/collector.go` `captureAndroidScreenshot()`
Android (scrcpy)	scrcpy-direct v3 wire protocol	`pkg/capture/android_capture.go`, `pkg/bridge/scrcpy/`

Platform	Method	File
Web	Playwright `page.screenshot()`	`pkg/evidence/collector.go` `captureWebScreenshot()`
Desktop (Linux)	X11 `import` or `xwd`	`pkg/evidence/collector.go` `captureDesktopScreenshot()`
Desktop (macOS)	`screencapture`	`pkg/capture/macos_capture.go`
Desktop (Windows)	`PrintWindow` / D3D	`pkg/capture/windows_capture.go`

8.2 Video Recording

- **FFmpeg-based**: `pkg/video/ffmpeg_recorder.go` — captures desktop via `ffmpeg -f x11grab` or `gdigrab`.
- **scrcpy-based**: `pkg/video/scrcpy.go` — streams Android screen via scrcpy server.
- **Session lifecycle**: `pkg/session/recorder.go` `StartVideo()` / `StopVideo()` manages per-platform video files.

8.3 Audio Recording

- `pkg/evidence/collector.go` `StartAudio()` / `StopAudio()`
- Uses FFmpeg or PulseAudio/ALSA capture.
- Configurable quality: `standard` (44.1kHz/16bit), `high` (48kHz/24bit), `ultra` (96kHz/32bit).
- Format: `wav` (lossless) or `flac`.

8.4 Evidence Organization

```
qa-results/
├── report.md
├── report.html
├── report.json
├── evidence/
│   ├── screenshot-step1-1714500000000.png
│   ├── screenshot-step2-1714500001000.png
│   ├── video-android-1714500000000.mp4
│   ├── video-web-1714500000000.mp4
│   ├── logcat-1714500000000.txt
│   ├── audio-1714500000000.wav
│   └── stacktrace-1714500000000.txt
└── tickets/
    ├── TICKET-001.md
    └── TICKET-002.md
```

Evidence Item tracking (`pkg/evidence/collector.go`):

```go
type Item struct {
    Type      Type           `json:"type"`      // screenshot | video | logcat | stacktrace | console_log | audio
    Path      string         `json:"path"`
```

```
    Platform  config.Platform `json:"platform"`
    Step      string         `json:"step,omitempty"`
    Timestamp time.Time      `json:"timestamp"`
    Size      int64          `json:"size"`
}
```

8.5 On-Demand Screenshot Requests

The architecture supports on-demand screenshots via multiple paths: 1. **Autonomous session**: `PlatformWorker` calls `executor.Screenshot(ctx)` during `PerformAction()`; screenshots are resized to 480 px max for LLM vision (`pkg/autonomous/screenshot.go`). 2. **Validation pipeline**: `pkg/validator` captures pre/post screenshots around each step. 3. **Evidence collector**: `pkg/evidence/collector.go` `CaptureScreenshot()` can be called directly by any package with a `context.Context`. 4. **Navigation engine**: `NavigationEngine` captures screenshots to determine `ScreenChanged`. 5. **Issue detector**: `IssueDetector` captures screenshots for LLM analysis.

---

9. Tests Within HelixQA (`tests/`)

9.1 Test Directory Structure

```
tests/
├── benchmark/
│   └── benchmark_test.go
├── e2e/
│   └── e2e_test.go
├── integration/
│   └── integration_test.go
├── security/
│   └── (security tests)
└── stress/
    └── (stress tests)
```

9.2 Per-Package Test Coverage

From `ARCHITECTURE.md`:

Package	Tests	Focus
`pkg/config`	Unit + edge	Validation, parsing, defaults
`pkg/detector`	Unit + platform	Android/Web/Desktop detection
`pkg/validator`	Unit + concurrent	Step validation, evidence
`pkg/reporter`	Unit + format	Markdown, HTML, JSON output
`pkg/orchestrator`	Unit + edge + integration + stress	Full pipeline, cancellation

Package	Tests	Focus
pkg/testbank	Unit + stress + benchmark	YAML loading, filtering
pkg/ticket	Unit + stress + benchmark	Markdown generation
pkg/evidence	Unit + stress + benchmark	Concurrent capture

Total: **235 tests**, all passing with `-race` flag.

9.3 Notable Test Files by Package

- pkg/autonomous/coordinator_test.go
- pkg/autonomous/phase_test.go
- pkg/autonomous/pipeline_test.go
- pkg/autonomous/executor_factory_test.go
- pkg/autonomous/http_executor_test.go
- pkg/autonomous/playwright_executor_test.go
- pkg/autonomous/real_executor_test.go
- pkg/autonomous/structured_executor_test.go
- pkg/autonomous/screenshot_test.go (implicit)
- pkg/autonomous/stagnation_test.go
- pkg/autonomous/bocpd_test.go
- pkg/autonomous/geo_probe_test.go
- pkg/autonomous/fallback_test.go
- pkg/autonomous/findings_bridge_test.go
- pkg/autonomous/retry_test.go
- pkg/autonomous/sanitize_test.go
- pkg/autonomous/adapters_test.go
- pkg/autonomous/bank_realbinary_test.go
- pkg/autonomous/allow_foreground_leave_test.go
- pkg/autonomous/foreground_parse_test.go
- pkg/navigator/engine_test.go
- pkg/navigator/executor_test.go
- pkg/navigator/api_executor_test.go
- pkg/navigator/cli_executor_test.go
- pkg/navigator/playwright_executor_test.go
- pkg/navigator/x11_executor_test.go
- pkg/navigator/state_test.go
- pkg/navigator/dual_screen_test.go
- pkg/navigator/clear_test.go
- pkg/issuedetector/detector_test.go
- pkg/session/recorder_test.go
- pkg/session/timeline_test.go
- pkg/session/video_test.go
- pkg/evidence/collector_test.go
- pkg/evidence/collector_stress_test.go
- pkg/evidence/annotator_test.go
- pkg/evidence/audio_test.go
- pkg/vision/detector_test.go
- pkg/vision/diff_test.go
- pkg/vision/integration_test.go
- pkg/vision/llm_ollama_test.go
- pkg/vision/ocr_paddle_test.go

- pkg/vision/ocr_tesseract_test.go
- pkg/testbank/manager_test.go
- pkg/testbank/manager_stress_test.go
- pkg/testbank/schema_test.go
- pkg/testbank/duplicate_test.go
- pkg/testbank/generator_test.go
- pkg/llm/adaptive_test.go
- pkg/llm/adaptive_enhanced_test.go
- pkg/llm/anthropic_test.go
- pkg/llm/astica_test.go
- pkg/llm/bridge_provider_test.go
- pkg/llm/consensus_test.go
- pkg/llm/cost_tracker_test.go
- pkg/llm/escalation_test.go
- pkg/llm/google_test.go
- pkg/llm/ollama_test.go
- pkg/llm/openai_test.go
- pkg/llm/phase_selector_test.go
- pkg/llm/prompt_optimizer_test.go
- pkg/llm/provider_test.go
- pkg/llm/providers_registry_test.go
- pkg/llm/ratelimiter_test.go
- pkg/llm/vision_ranking_test.go
- pkg/regression/visual_test.go
- pkg/regression/pixelmatch_test.go
- pkg/regression/deltae_test.go
- pkg/regression/report_test.go
- pkg/reporter/reporter_test.go
- pkg/validator/validator_test.go
- pkg/detector/detector_test.go
- pkg/config/config_test.go
- pkg/controller/controller_test.go
- pkg/memory/store_test.go
- pkg/memory/findings_test.go
- pkg/memory/cognitive_test.go
- pkg/memory/coverage_test.go
- pkg/memory/knowledge_test.go
- pkg/memory/sessions_test.go
- pkg/planning/planner_test.go
- pkg/planning/ranker_test.go
- pkg/planning/reconciler_test.go
- pkg/planning/androidtv_channels_framework_test.go
- pkg/planning/channels_support_test.go
- pkg/learning/codebase_test.go
- pkg/learning/git_test.go
- pkg/learning/knowledge_test.go
- pkg/learning/manifest_test.go
- pkg/learning/platform_features_test.go
- pkg/learning/reader_test.go
- pkg/performance/collector_test.go
- pkg/reproduce/reproducer_test.go
- pkg/replay/buffer_test.go
- pkg/streaming/scrcpy_rtsp_bridge.go (live tests)
- pkg/streaming/webrtc_server_test.go

- `pkg/streaming/webrtc_example_test.go`
- `pkg/video/scrcpy_test.go`
- `pkg/video/scrcpy_live_test.go`
- `pkg/video/frames_test.go`
- `pkg/visionnav/provider_test.go`
- `pkg/visionnav/visionnav_test.go`
- `pkg/visual/comparator_test.go`
- `pkg/audio/stream_test.go`
- `pkg/audio/tesseract_client_test.go`
- `pkg/audio/whisper_client_test.go`
- `pkg/nexus/adapter_test.go`
- `cmd/helixqa/replay_test.go`
- `cmd/helixqa/nexus_adapters_test.go`

---

10. Integration Dependencies (`.gitmodules`)

10.1 Submodule Listing

HelixQA depends on **25+ open-source tools** vendored as Git submodules under `tools/opensource/`:

Submodule	Path	Purpose
scrcpy	`tools/opensource/scrcpy`	Android screen mirroring / control
allure2	`tools/opensource/allure2`	Test reporting framework
leakcanary	`tools/opensource/leakcanary`	Android memory leak detection
docker-android	`tools/opensource/docker-android`	Android in Docker
appium	`tools/opensource/appium`	Mobile test automation
midscene	`tools/opensource/midscene`	Web UI automation
mem0	`tools/opensource/mem0`	Memory layer for LLMs
moondream	`tools/opensource/moondream`	Tiny vision-language model

Submodule	Path	Purpose
ui-tars	`tools/opensource/ui-tars`	UI understanding model
perfetto	`tools/opensource/perfetto`	Android system tracing
chroma	`tools/opensource/chroma`	Vector database
shortest	`tools/opensource/shortest`	AI end-to-end testing
marker	`tools/opensource/marker`	PDF / document OCR
kiwi-tcms	`tools/opensource/kiwi-tcms`	Test case management
testdriverai	`tools/opensource/testdriverai`	AI test automation
stagehand	`tools/opensource/stagehand`	Browser automation
unstructured	`tools/opensource/unstructured`	Document parsing
redroid	`tools/opensource/redroid`	Android in Docker (remote)
signoz	`tools/opensource/signoz`	Observability platform
docling	`tools/opensource/docling`	Document understanding
llama-index	`tools/opensource/llama-index`	LLM data framework
appcrawler	`tools/opensource/appcrawler`	App crawler
browser-use	`tools/opensource/browser-use`	Browser-use AI

Submodule	Path	Purpose
skyvern	`tools/opensource/skyvern`	AI web automation
anthropic-quickstarts	`tools/opensource/anthropic-quickstarts`	Anthropic examples
ui-tars-desktop	`tools/opensource/ui-tars-desktop`	Desktop UI-TARS

Test app submodule: - `rest-demo` → `tools/test-apps/rest-demo` (`nicehash/rest-clients-demo`)

10.2 Sibling Dependencies

From `ARCHITECTURE.md` and `go.mod` imports:

Module	Path	Role
`digital.vasic.challenges`	`../Challenges`	Challenge execution engine, runner, bank loader, reporter
`digital.vasic.containers`	`../Containers`	Container orchestration, runtime, lifecycle
`digital.vasic.llmorchestrator`	External	LLM agent pool, CLI adapters
`digital.vasic.visionengine`	External	GoCV + LLM vision, NavigationGraph
`digital.vasic.docprocessor`	External	Feature maps, coverage tracking

---

11. Key Design Patterns

1. **Composition over reimplementation**: HelixQA imports *Challenges* types directly (`challenge.Definition`, `bank.Bank`, `report.Reporter`). No wrapper types.
2. **Functional options pattern**: All constructors use `WithX()` options for clean dependency injection and testing.
3. **CommandRunner interface**: Abstracts command execution (`adb`, `npx`, `xte`) behind an interface, enabling full test coverage without real devices.
4. **Platform-agnostic orchestration**: The orchestrator runs the same pipeline for all platforms. Platform-specific behavior is encapsulated in `pkg/detector`, `pkg/evidence`, `pkg/navigator`.
5. **Evidence-first reporting**: Every failure includes evidence (screenshots, logs, traces). Tickets are self-contained for AI pipeline consumption.

6. **Dual-model architecture**: Vision models analyze screenshots; chat models generate test plans, write reports, reason about findings.
7. **Anti-bluff compliance**: Article XI §11.5 — explicit `_skip` support with `_skip_reason`; Article XI §11.9 — user-mandate forensic anchor; CONST-035 anti-bluff guard.

---

12. Notable Architectural Decisions

- **BOCPD for stagnation**: `pkg/autonomous/stagnation.go` uses Bayesian Online Change-Point Detection (Adams & MacKay 2007) over dHash Hamming-distance stream for per-frame UI change probability — no need to wait 10 seconds for "UI is stuck" heuristic.
- **Geo-restriction probing**: `pkg/autonomous/geo_probe.go` probes app content endpoints before playback; marks `GEO_RESTRICTED` so tests `SKIP` rather than `FAIL`.
- **Android TV competing-app guard**: `PipelineConfig.CompetingAppPackages` force-stops foreign apps before testing to prevent DPAD_ENTER from handing control to wrong app.
- **CSRF preflight in HTTP executor**: `HTTPExecutor` automatically handles CSRF token lifecycle for mutating API requests.
- **Token caching**: `HTTPExecutor` caches admin session tokens across steps to avoid N login round-trips.
- **Blank screenshot detection**: `IsBlankScreenshot()` samples 9x9 grid (81 points) and checks per-channel range; catches faint widgets while rejecting truly blank frames.
- **Screenshot resizing**: Screenshots sent to LLM are downscaled to max 480 px wide via nearest-neighbor for fast CPU inference.
- **Distributed vision**: `PipelineConfig.VisionHosts` probes multiple hosts via SSH, selects strongest model fitting combined resources, activates llama.cpp RPC if needed.
- **Constitution compliance**: `CONSTITUTION.md` governs architecture decisions; `CONST-035` anti-bluff; `Article XI §11.9` forensic anchor; `Article XI §11.5` honest skip mandate.

Document generated from live source analysis of `https://github.com/HelixDevelopment/HelixQA` *(main branch, ~570 commits, commit* `0bca023`*).*